hashlock.

2024

Security Audit

Silverswap (DCA)

June 2024

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

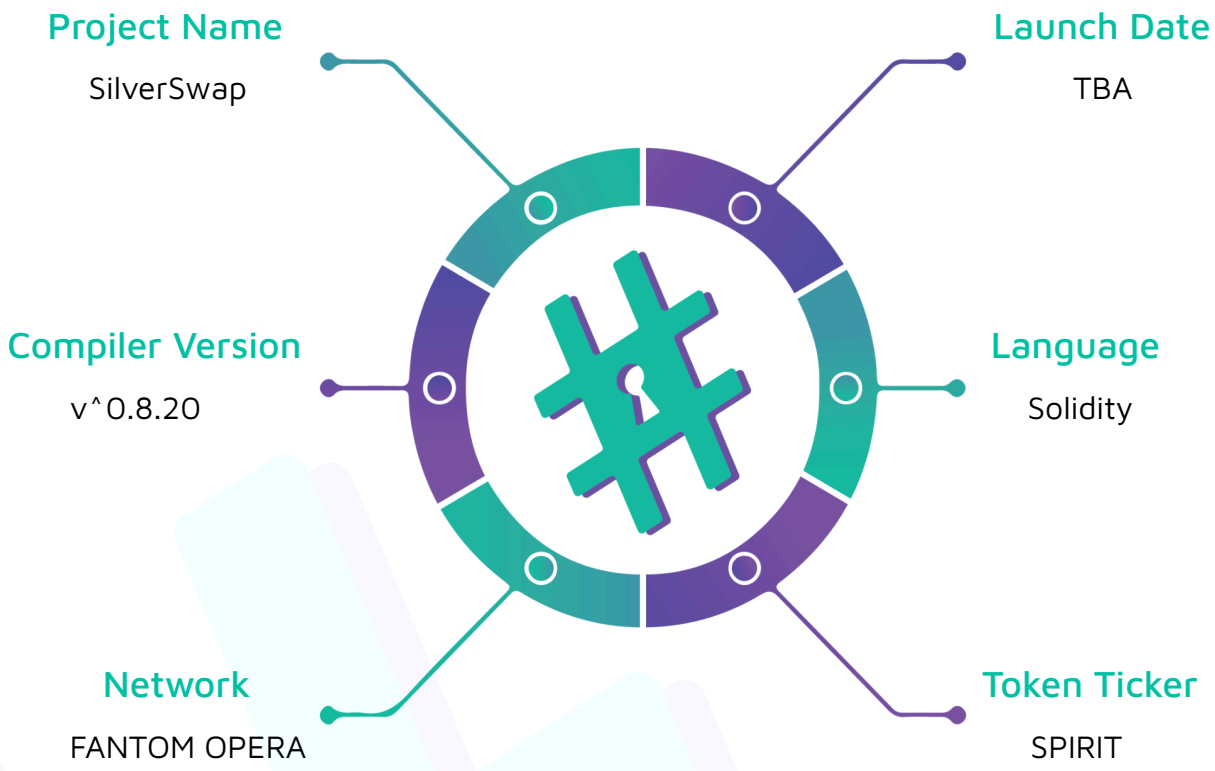
The SilverSwap team partnered with Hashlock to conduct a security audit of their DCA protocol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts were secure.

Project Context

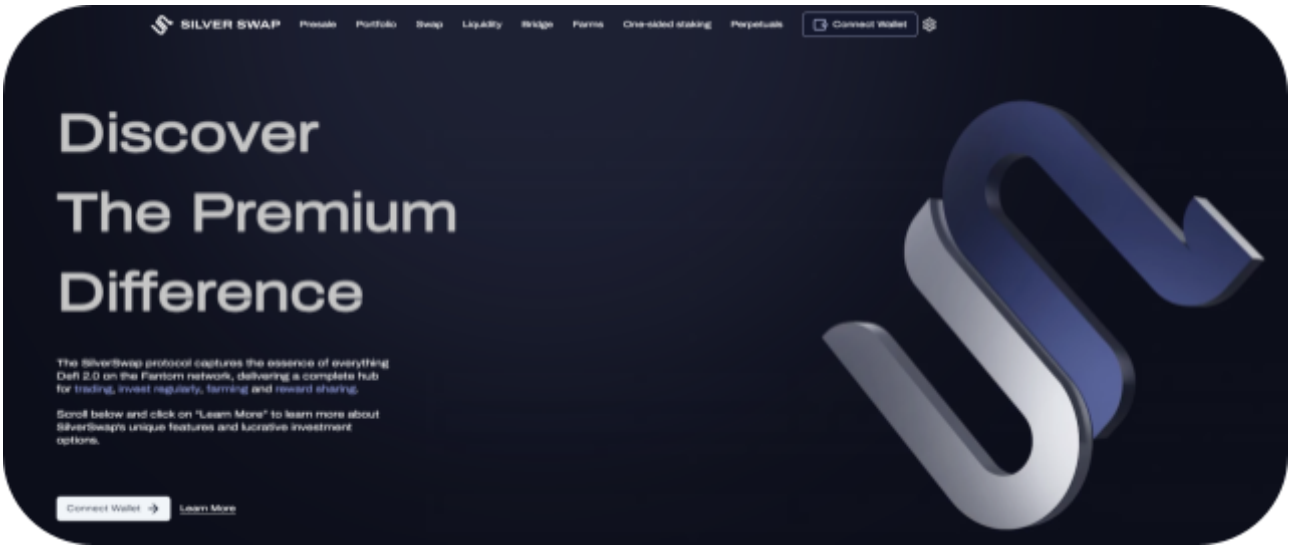
SilverSwap.io is a decentralized exchange (DEX) that enables users to trade and swap cryptocurrencies in a secure, non-custodial manner. The platform is built to facilitate decentralized trading without the involvement of intermediaries, offering users full control over their digital assets. By operating on blockchain technology, SilverSwap ensures transparency and security in transactions, while promoting a peer-to-peer trading model. Users can participate in liquidity pools, where they provide liquidity in exchange for potential rewards, helping to maintain the platform's fluidity and enabling seamless swaps of various digital assets.

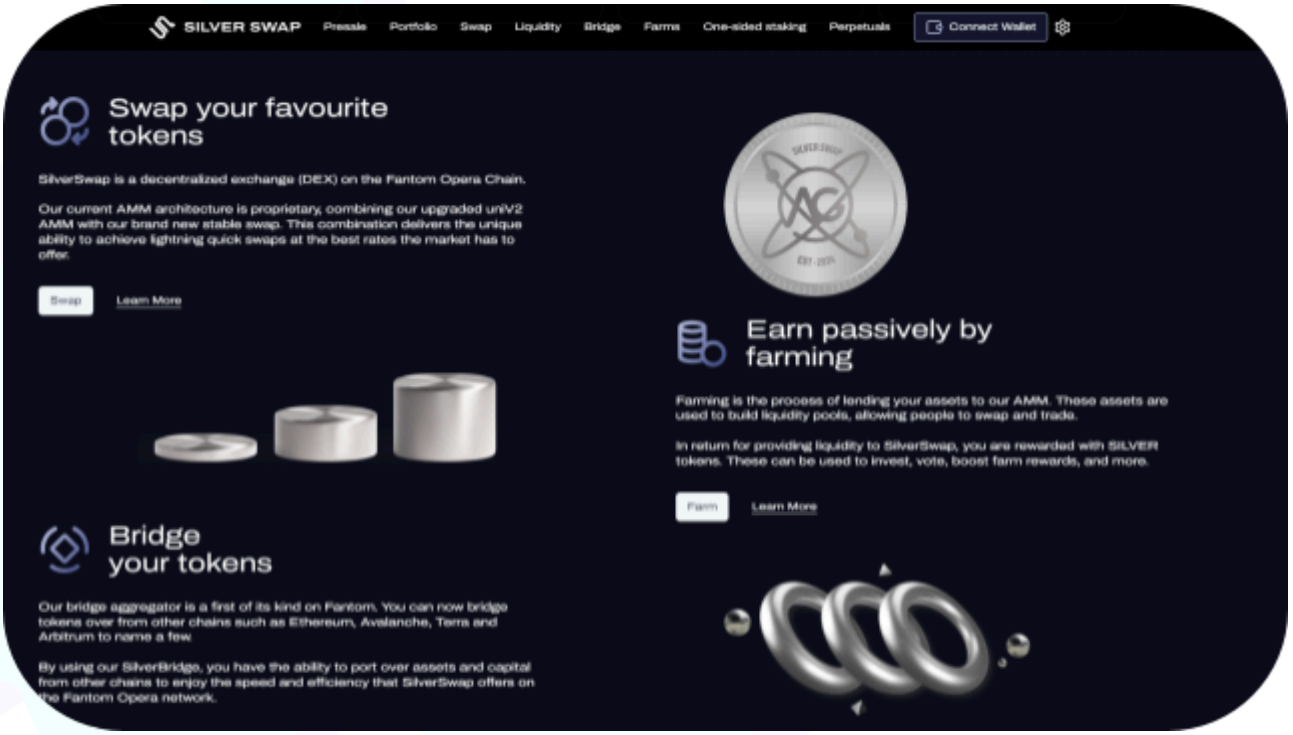
Project Name: SilverSwap
Compiler Version: ^0.8.20
Website: https://silverswap.io/
Logo:

Visualised Context:



Project Visuals:





Audit scope

We at Hashlock audited the solidity code within the SilverSwap project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	**Template Protocol Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**May, 2024**
Contract 1	AutomateReady.sol
Contract 1 MD5 Hash	eaa828967bf3dbbd8d7e39185e7c9b74
Contract 2	AutomateTaskCreator.sol
Contract 2 MD5 Hash	8dd21e976edb1e2b8f04c91ad330efa5
Contract 3	Types.sol
Contract 3 MD5 Hash	07e0dca63de1216683b0e05408653b38
Contract 4	IOPSProxy.sol
Contract 4 MD5 Hash	d9043397a1b9bb70465d35081b25cade
Contract 5	IParaswap.sol
Contract 5 MD5 Hash	60ff120606d5bde75054eafed0a01bde
Contract 6	TransferHelper.sol
Contract 6 MD5 Hash	1945c6c248206f433d9a8c438f30c2cb
Contract 7	Utils.sol
Contract 7 MD5 Hash	2321a8c2462bb96e9facd058fe0c0134
Contract 8	DcaApprover.sol
Contract 8 MD5 Hash	b47c6cf79f1622c15771b7ee0b64eb90

Contract 9	SpiritDCA.sol
Contract 9 MD5 Hash	0cdb87c2b08e98cc80859b8ee17915a6

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all have correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that need to be addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

All vulnerabilities initially identified are currently resolved or acknowledged.

Hashlock found:

3 High severity vulnerabilities

0 Medium severity vulnerabilities

6 Low severity vulnerabilities

6 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour

AutomateReady.sol
- When inherited allows contracts to:
  - Make synchronous fee payments
  - Have call restrictions for functions to be automated
	Contract achieves this functionality.

AutomateTaskCreator.sol - When inherited allows contracts to: - Be a task creator and create tasks	**Contract achieves this functionality.**
Types.sol - Contract that defines enumerations and structs	**Contract achieves this functionality.**
TransferHelper.sol - Library that defines transfer functions	**Contract achieves this functionality.**
Utils.sol - Utility library that defines structs and functions.	**Contract achieves this functionality.**
DcaApprover.sol - Contract that gets deployed for each new order users create and handles swaps on the DEX	**Contract achieves this functionality.**

SpiritDCA.sol
- Main contract that users interact with:
- Deploys a new DcaApprover.sol for each order
- Allows the owner to withdraw fees.
- Allows users to:
  - Create new orders
  - Edit/stop/restart their orders
	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the SilverSwap project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring is required.

The code is not well commented and does not follow best practice nat-spec styling.

Audit Resources

We were given the SilverSwap project smart contract code in the form of Github access.

As mentioned above, code parts are not well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The README file is helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

High

[H-01] SpiritDCA.sol::createTask() - Precision loss when calculating for `execData amountIn`

Description

In the `createTask()` function, `execData` takes necessary swap data and saves it in strings. This data is then used in the `AutomateTaskCreator.sol::_createTask()` function. However due to division before multiplication, the saved string value of `amountIn` will be different from the intended result.

Vulnerability Details

SilverSwap takes a 1% fee for orders, this is why in the `createTask()` function, 99% of `amountIn` is calculated and stored in `execData` to be used in the swap as shown in the code snippet below.

```
function createTask(uint256 id) private {

     require(ordersById[id].taskId == bytes32(""), 'Task already created.');

     bytes memory execData = abi.encode(

        ...

        Strings.toString((ordersById[id].amountIn / 100) * 99),

        ...
}
```

Here, during the calculation of 99% of `amountIn` value, division is done before multiplication which results in precision loss, this is because the division operation in Solidity performs integer division, discarding any remainder.

Proof of Concept

To see the loss of precision compare the results of these two functions:

```
uint256 amountIn = 987987987;

    function divisionBeforeMultiplication() public view returns (uint256){

        return (amountIn / 100) * 99;

    }

    function divisionAfterMultiplication() public view returns (uint256){

        return (amountIn * 99) / 100;

    }
```

While the first function gives the result of `978108021`, second function will give `978108107` which is closer to the actual 99% of `978978978` which is `978108107.13`

Impact

Precision loss causes incorrect calculation of `amountIn` that is sent to the swap. Users will lose more tokens than they should based on the 1% fee that SilverSwap takes.

Recommendation

Do the division after the multiplication as shown below.

```
function createTask(uint256 id) private {

     require(ordersById[id].taskId == bytes32(""), 'Task already created.');

     bytes memory execData = abi.encode(

        ...

        Strings.toString((ordersById[id].amountIn * 99) * 100),

        ...
}
```

Status

Resolved

[H-02] SpiritDCA.sol::createOrder() - Incorrect boundaries for amountIn results in protocol getting no fees and wrong amount to be set in execData.

Description

In the `createOrder()` function, `amountIn` is enforced to be more than 0. However, multiple calculations will give incorrect results when `amountIn` is less than 100 and greater than 0.

Vulnerability Details

Due to the fee calculation in `_executeOrder`, any `amountIn` that is less than 100 and more than 0 will result in the protocol not getting any fees due to the division. `execData` will also save `0` to the string that is sent to the swap.

```
function _executeOrder(uint id, paraswapArgs memory dcaArgs) private {

     ...

     uint256 fees = ordersById[id].amountIn / 100;

      ...

function createTask(uint256 id) private {

     require(ordersById[id].taskId == bytes32(""), 'Task already created.');

     bytes memory execData = abi.encode(

        ...

        Strings.toString((ordersById[id].amountIn / 100) * 99),

        ...
}
```

For example, a user entering 99 as `amountIn` will result in `fees` and the encoded string for 99% of `amountIn` in `execData` to be 0.

Proof of Concept

To see the loss of precision, run the following function:

```
uint256 amountIn = 99;

    function calculateFeesAndExecDataAmount() public view returns (uint256, uint256){

        uint256 fees = amountIn / 100;

        uint256 execDataAmount = (amountIn / 100) * 99;

        return (fees, execDataAmount);

    }
```

Function will return `fees` and `execDataAmount` as 0.

Impact

A user creating this order will result in a user executing one order with 99 tokens while paying SilverSwap protocol no fees. Following executions of this order will be done for 0 tokens due to how `execData` is saved even though the `createOrder()` function has 0 check implemented.

Recommendation

Adjust the boundary for `amountIn` in `createOrder()` function as shown below.

```
function createOrder(address tokenIn, address tokenOut, uint256 amountIn, uint256
amountOutMin, uint256 period, paraswapArgs memory dcaArgs) public {

     require(period >= 1 days, 'Period must be greater than 1 day.');

     require(amountIn >= 100, 'AmountIn must be greater than 99.');

     ...

}
```

Status

Resolved

[H-03] SpiritDCA.sol::executeOrder() - Lack of access control

Description

Any malicious attacker can call `executeOrder()` function with the parameters they want to enter.

Vulnerability Details

Consider the scenario where an attacker will call `executeOrder()` function with the `id` of any other user. Attacker here can enter their desired `ftmSwapArgs` into this function.

```
if (isFtmSwap)
     {
     uint256 gelatoFees = 0;
     if (!isSimpleDataEmpty(ftmSwapArgs.simpleData)) {
            gelatoFees = ftmSwapArgs.simpleData.fromAmount;
            ordersById[id].amountIn -= gelatoFees;
     } else if (!isSellDataEmpty(ftmSwapArgs.sellData)) {
            gelatoFees = ftmSwapArgs.sellData.fromAmount;
            ordersById[id].amountIn -= gelatoFees;
     } else if (!isMegaSwapSellDataEmpty(ftmSwapArgs.megaSwapSellData)) {
            gelatoFees = ftmSwapArgs.megaSwapSellData.fromAmount;
            ordersById[id].amountIn -= gelatoFees;
     }
     SpiritDcaApprover(ordersById[id].approver).transferGelatoFees(gelatoFees);
     ERC20(ordersById[id].tokenIn).approve(address(proxy), gelatoFees);
```

In the code snippet above, notice that the attacker can set the `ftmSwapArgs.simpleData.fromAmount` to any amount that is <= `amountIn` to

avoid underflow. Here, assume that the attacker set the `gelatoFees` to the max amount they can.

This function will then call the `DcaApprover.sol::transferGelatoFees()` function with the amount that the attacker has set.

```
function transferGelatoFees(uint256 feesAmount) public {

        require(msg.sender == dca, 'Only DCA can execute order.');

        Order memory order = ISpiritDCA(dca).ordersById(id);

       require(block.timestamp - order.lastExecution >= order.period, 'Period not elapsed.');

        require(!order.stopped, 'Order is stopped.');

        TransferHelper.safeTransferFrom(tokenIn, user, dca, feesAmount);

    }
```

Notice that the `transferGelatoFees()` function shown above will transfer the amount set by the attacker, from the user into the `SpiritDCA.sol` contract.

Lack of access control here causes similar, dangerous vulnerabilities when `dcaArgs` set by the attacker is put into `_executeOrder()` function, or when the `executeOrder()` function calls `proxy.simpleSwap()`.

Impact

An attacker will cause a user to lose funds.

Recommendation

Implement access control into the `executeOrder()` function so only the automation contract or the owner of the order can call it. For example: the `AutomateReady.sol::onlyDedicatedMsgSender()` modifier. More detailed approach on this in Gelato docs.

Additionally, implement restrictions for `ftmSwapArgs` and `dcaArgs`.

Status

Resolved

Low

[L-01] SpiritDCA.sol::getApproveAddress - View function does not return users approver address.

Description

`getApproveAddress()` function is not used in any other function so It is assumed that this function should be returning a users approver address. However, this function will not return users approver address.

Recommendation

Update the function so it returns users approver addresses.
Potentially remove the function to save gas.

Status

Acknowledged

[L-02] SpiritDCA.sol::createOrder() - Function should have zero check for `amountOutMin`

Description

`amountOutMin` value that is entered by the user in the `createOrder()` function, protects the user against slippage. However, this function lacks 0 checks. A user can accidentally leave this blank while entering the parameters and send a swap order instantly (`_executeOrder()` is called when creating an order) without any slippage protection.

Recommendation

Implement the following if check in the `createOrder()` function:

```
function createOrder(address tokenIn, address tokenOut, uint256 amountIn, uint256 amountOutMin, uint256 period, paraswapArgs memory dcaArgs) public {

     if(amountOutMin = 0){

        revert;

     }
```

Status

Resolved

[L-03] SpiritDCA.sol::_executeOrder() - Return value of `transfer()` not checked

Description

Not all ERC20 implementations `revert()` when there's a failure in `transfer()` or `transferFrom()`. The function signature has a boolean return value and they indicate errors that way instead. By not checking the return value, operations that should have been marked as failed, may potentially go through without actually transferring anything.

Recommendation

To ensure the reliability and security of token transfers in your smart contract, it's crucial to check the return values of the `transfer()` and `transferFrom()` functions. These functions often return a boolean value indicating the success or failure of the transfer operation. By checking this return value, you can accurately determine whether the transfer was successful and handle any potential errors or failures accordingly.

Status

Resolved

[L-04] Contracts - Using >/>= without specifying an upper bound in version pragma is unsafe

Description

There will be breaking changes in future versions of solidity, and at that point your code will no longer be compatible. While you may have the specific version to use in a configuration file, others that include your source files may not.

Recommendation

To ensure the long-term compatibility and reliability of your Solidity code, avoid using version pragma statements with > or >= without specifying an upper bound. Instead, explicitly define the maximum compatible Solidity version. This helps safeguard your code against potential breaking changes in future Solidity releases, ensuring it remains compatible for all users.

Status

Resolved

[L-05] Contracts - Use `Ownable2Step` rather than `Ownable`

Description

`Ownable2Step` and `Ownable2StepUpgradeable` prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using `Ownable2Step` or `Ownable2StepUpgradeable` from OpenZeppelin Contracts to enhance the security of your contract ownership management. These contracts prevent the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively

accept ownership via a contract call. This two-step ownership transfer process adds an additional layer of security to your contract's ownership management.

Status

Resolved

[L-06] Contracts - Unsafe ERC20 operation `approve()`

Description

Approve call do not handle non-standard erc20 behaviour. Use `safeApprove` instead of `approve`. Some token contracts do not return any value and some token contracts revert the transaction when the allowance is not zero.

Recommendation

Use `safeApprove` instead of `approve`

Status

Resolved

Gas

[G-01] Contracts - Revert string size optimization

Description

Shortening the revert strings to fit within 32 bytes will decrease deployment time and decrease runtime Gas when the revert condition is met. Revert strings that are longer than 32 bytes require at least one additional `mstore`, along with additional overhead to calculate memory offset, etc.

Recommendation

To optimise Gas usage in your Solidity contract, it is recommended to keep revert strings as short as possible and to ensure that they fit within 32 bytes. It is possible to use abbreviations or simplified error messages to keep the string length short. Doing so can reduce the amount of Gas used during deployment and runtime when the revert condition is met.

Status

Resolved

[G-02] Contracts - Custom errors in Solidity for Gas Efficiency

Description

Using custom errors instead of require statements with string messages, the gas consumption can be significantly reduced, leading to more gas-efficient contracts.

Recommendation

It is recommended to use custom errors instead of revert strings to reduce gas costs, especially during contract deployment. Custom errors can be defined using the error keyword and can include dynamic information.

Status

Acknowledged

[G-03] SpiritDCA.sol::`_executeOrder()` - Repeated function calls in `_executeOrder()`

Description

`_executeOrder()` function does multiple calls to the same functions such as `isSimpleDataEmpty`. Avoid repeated calls to these functions to save gas.

Recommendation

Consider calling `isSimpleDataEmpty` and other repeated functions once and assign them to memory variables.

Status

Resolved

[G-04] Contracts - Require statements used multiple times can be set as a modifier.

Description

Implementing modifiers for require statements that are used multiple times in different functions will make the code more readable and can save some gas.

Recommendation

Instead of using the same require statement in multiple functions, assign it to modifiers. For example: `require(ordersById[id].user == msg.sender`.

Status

Resolved

[G-05] Contracts - Missing constant modifier for state variables

Description

State variables that are never reassigned after their initial assignment should typically be declared with the `constant` modifier. This ensures that these variables remain unmodified and communicates their unchanging nature. Using the `constant` modifier also offers potential gas savings, as accessing these variables does not require any storage operations.

Recommendation

Declare state variables that are not reassigned after initialization with the `constant` modifier in Solidity. This clarifies their unchanging nature, improves code readability, and saves gas by avoiding storage operations when accessing these variables.

Status

Resolved

[G-06] Contracts - Avoid using state variables directly in `emit` for gas efficiency

Description

In Solidity, emitting events is a common way to log contract activity and changes, especially for off-chain monitoring and interfacing. However, using state variables directly in emit statements can lead to increased gas costs. Each access to a state variable incurs gas due to storage reading operations. When these variables are used directly in emit statements, especially within functions that perform multiple operations, the cumulative gas cost can become significant. Instead, caching state variables in memory and using these local copies in emit statements can optimise gas usage.

Recommendation

To optimise gas efficiency, cache state variables in memory when they are used multiple times within a function, including in emit statements.

Status

Resolved

Centralisation

The SilverSwap project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the SilverSwap project seems to have a sound and well tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


#Hashlock.